Tupã: Uma Linguagem de Domínio Específico em Português para Programação de Microcontroladores ESP32

Michel Lutegar
Engenharia de Computação
IBMEC
Rio de Janeiro, Brasil
email@aluno.ibmec.edu.br

André Costa
Engenharia de Computação
IBMEC
Rio de Janeiro, Brasil
email@aluno.ibmec.edu.br

Felipe Dutrain
Engenharia de Computação
IBMEC
Rio de Janeiro, Brasil
202402011359@alunos.ibmec.edu.br

Guilherme Lopes
Engenharia de Computação
IBMEC
Rio de Janeiro, Brasil
202401632473@aluno.ibmec.edu.br

***Resumo*—Programar microcontroladores ainda exige domínio de C++, de bibliotecas específicas e da configuração manual de pinos, o que representa uma barreira significativa para iniciantes, estudantes e entusiastas de eletrônica. Este artigo apresenta o Tupã, uma linguagem de domínio específico com sintaxe em português e eventos de hardware como construções de primeira classe, voltada para a placa ESP32. O compilador, implementado em Python, percorre as quatro fases clássicas de compilação, análise léxica, análise sintática, análise semântica e geração de código, e produz um programa Arduino em C++ destinado à compilação e à gravação na placa pela Arduino CLI, etapa que ainda é manual. A análise semântica verifica o uso correto dos módulos de hardware antes da gravação e emite mensagens de erro didáticas em português. Em um exemplo de controle de acesso por cartão NFC, um programa de 12 linhas de código em Tupã gera 100 linhas de código C++, cerca de 8,3 vezes mais, sem que o usuário precise escrever inclusões de bibliotecas, definições de pinos ou as funções de configuração e laço principal. Os programas gerados compilam com a Arduino CLI, e um teste de bancada com firmware escrito à mão validou o protótipo com ESP32-WROVER, leitor RC522 e sensor infravermelho de obstáculo. O projeto encontra-se em desenvolvimento, com as quatro fases do compilador implementadas e a gravação do código gerado na placa ainda pendente.**

***Index Terms*—compiladores, linguagem de domínio específico, sistemas embarcados, ESP32, Arduino, ensino de programação**

I. Introdução

A popularização de placas de baixo custo, como Arduino e ESP32, aproximou estudantes e entusiastas do desenvolvimento de sistemas embarcados. Ainda assim, o caminho entre a ideia e o hardware funcionando costuma ser longo: mesmo uma aplicação simples, como acender um LED quando um cartão válido é aproximado de um leitor, exige incluir bibliotecas, definir pinos, inicializar periféricos e organizar o código nas funções `setup()` e `loop()` do ambiente Arduino. Para quem não domina C++, a configuração do hardware frequentemente consome mais tempo do que a lógica da aplicação.

Somam-se a isso a barreira do idioma, já que palavras-chave, bibliotecas e documentação estão majoritariamente em inglês, e o fato de que ferramentas educacionais existentes ou dependem de hardware fechado, ou trabalham com blocos visuais que não evoluem para uma linguagem textual.

Este trabalho propõe o *Tupã*, uma linguagem de domínio específico (DSL, do inglês *domain-specific language*) com sintaxe em português, cujo compilador traduz programas de alto nível para C++ compatível com o ecossistema Arduino e executável no ESP32. A premissa central é que o usuário descreva *o que* o dispositivo deve fazer, e não *como* o hardware deve ser configurado, delegando ao compilador o trabalho de baixo nível.

As principais contribuições são:

- o projeto de uma linguagem textual em português com eventos de hardware de primeira classe, como `ao botao(1) { ... };`
- um compilador completo, implementado em Python, que percorre as fases de análise léxica, sintática e semântica e de geração de código;
- uma análise semântica orientada ao hardware, que detecta erros antes da gravação na placa e produz mensagens didáticas em português;
- uma avaliação preliminar que compara o código escrito em Tupã com o código C++ equivalente gerado.

O restante do artigo está organizado da seguinte forma: a Seção II discute trabalhos relacionados; a Seção III descreve a linguagem; a Seção IV detalha a arquitetura do compilador; a Seção V apresenta resultados preliminares; e a Seção VI conclui e aponta trabalhos futuros.

II. Trabalhos Relacionados

O *Arduino IDE* [2] consolidou-se como a principal porta de entrada para a programação de microcontroladores. Sua linguagem, um subconjunto de C++ acompanhado de bibliotecas, é poderosa e madura, mas exige do usuário conhecimento de C++, do mapeamento de pinos e das APIs de cada periférico.

O *Microsoft MakeCode* [4] e o *Blockly* [5] oferecem ambientes de programação por blocos voltados ao ensino. Embora reduzam a barreira inicial, não geram código Arduino nativo e não oferecem ao estudante uma linguagem textual real para a qual ele possa migrar.

O *Flipper Zero* [6] inspira o conceito de um dispositivo multifuncional para NFC, RFID e infravermelho, porém trata-se de hardware fechado e de custo elevado, sem foco em linguagem ou em ensino.

No contexto brasileiro, o *Portugol Studio* [7] mostrou o valor pedagógico de uma linguagem com palavras-chave em português para o ensino de algoritmos, mas não é voltado a sistemas embarcados.

O Tupã ocupa a lacuna entre essas abordagens: uma linguagem textual em português, que compila para C++/Arduino nativo e executa no ESP32, pensada para *makers* e para o ensino de sistemas embarcados.

III. A Linguagem Tupã

A. Princípios de Projeto

O Tupã foi projetado segundo três princípios: (i) *sintaxe em português*, com palavras-chave como `se`, `senao`, `inteiro` e `verdadeiro`; (ii) *eventos de hardware de primeira classe*, em que reações a estímulos físicos são construções da própria linguagem, e não chamadas de biblioteca; e (iii) *abstração de periféricos por módulos*, como `nfc`, `led` e `ir`, cujos detalhes de pinagem e inicialização ficam a cargo do compilador.

B. Exemplo Canônico

A Listagem 1 apresenta um controle de acesso por cartão: ao pressionar o botão 1, o dispositivo lê um cartão NFC e acende o LED verde se o cartão for válido, ou o vermelho caso contrário.

```
dispositivo controle {
    inteiro tentativas = 3;

    ao botao(1) {
        # le um cartao NFC
        cartao = nfc.ler();

        se cartao.valido == verdadeiro {
            led.verde();
            esperar(1500);
        } senao {
            led.vermelho();
        }
    }
}
```

Listagem 1. Controle de acesso por cartão NFC em Tupã (`controle_nfc.tupa`).

Um programa Tupã é composto por um bloco `dispositivo`, que pode declarar variáveis de estado e um ou mais tratadores de evento introduzidos pela palavra-chave `ao`. Dentro de um tratador, o usuário utiliza estruturas de controle e chamadas a métodos dos módulos de hardware.

C. Léxico

O vocabulário da linguagem é composto por 44 tipos de *token*: 43 enviados ao analisador sintático mais o marcador sintético de fim de entrada (`TK_EOF`). Espaços e comentários de linha, iniciados por `#`, são reconhecidos e descartados, e por isso não entram na contagem. Os nomes de evento `iniciar`, `obstaculo` e `cartao` não são *tokens* próprios: chegam como identificadores e são conferidos na análise semântica, o que mantém `cartao` livre como nome de variável. A Tabela I apresenta a distribuição por categoria.

Tabela I
Contagem de *tokens* da linguagem Tupã por categoria

Categoria	Qtde	Exemplos
Palavras reservadas	12	`dispositivo`, `ao`, `se`, `senao`, `repetir`, `inteiro`
Hardware e comandos	6	`nfc`, `ir`, `led`, `botao`, `esperar`, `escrever`
Identificador	1	`cartao`
Literais	3	inteiro, decimal, texto
Operadores	14	`==`, `=`, `&&`, `+`
Delimitadores	7	`{ }`, `( )`, `;`, `.`
Enviados ao parser	43	
Fim de entrada	1	`TK_EOF`
Total	**44**	

D. Gramática

A Fig. 1 apresenta a gramática, em notação EBNF, que o analisador sintático efetivamente aceita. Além de `ao botao(N)`, há os eventos `ao iniciar` (uma vez, na partida), `ao obstaculo` (mudança do sensor infravermelho) e `ao cartao` (cartão novo lido, com `cartao.uid` e `cartao.valido`), e o comando `escrever`, que envia um valor ao Monitor Serial. As regras de expressão estão agrupadas por nível de precedência, da menor para a maior.

IV. Arquitetura do Compilador

O compilador do Tupã é implementado em Python 3 e segue a arquitetura clássica de compiladores [1], organizada em quatro fases encadeadas, ilustradas na Fig. 2. A saída é um arquivo `.ino`, que deve então ser compilado e gravado no ESP32 pela Arduino CLI [3]; essa etapa não é executada pelo compilador.

A. Análise Léxica

O analisador léxico converte o texto-fonte em uma sequência de *tokens*, registrando linha e coluna de cada um para uso nas mensagens de erro das fases seguintes. A técnica é a de expressões regulares: uma única expressão mestre reúne todas as alternativas como grupos nomeados e é aplicada de forma iterativa sobre o texto, o que dá a cada *token* o grupo que casou primeiro. A ordem das alternativas é significativa, de modo que

```
programa  ::= "dispositivo" ID "{" item* "}" EOF
item      ::= evento | comando
evento    ::= "ao" "botao" "(" INT ")" bloco
            | "ao" nome_evento bloco
nome_evento ::= "iniciar" | "obstaculo" | "cartao"
bloco     ::= "{" comando* "}"
comando   ::= decl_var | se | repetir | esperar
            | escrever | retornar | atrib_ou_chamada
decl_var  ::= tipo ID ("=" expr)? ";"
tipo      ::= "inteiro" | "decimal" | "texto" | "booleano"
se        ::= "se" expr bloco ("senao" bloco)?
repetir   ::= "repetir" "(" expr ")" bloco
esperar   ::= "esperar" "(" expr ")" ";"
escrever  ::= "escrever" "(" expr ")" ";"
retornar  ::= "retornar" expr? ";"
atrib_ou_chamada ::= expr ("=" expr)? ";"
expr      ::= e ("||" e)*
e         ::= igualdade ("&&" igualdade)*
igualdade ::= comparacao (("==" | "!=") comparacao)*
comparacao ::= termo ((">" | "<" | ">=" | "<=") termo)*
termo     ::= fator (("+" | "-") fator)*
fator     ::= unaria (("*" | "/") unaria)*
unaria    ::= ("!" | "-") unaria | posfixo
posfixo   ::= primario ("." ID | "(" args? ")")*
args      ::= expr ("," expr)*
primario  ::= INT | FLOAT | STRING | "verdadeiro" | "falso"
            | "nfc" | "ir" | "led" | "botao" | ID
            | "(" expr ")"
```

Figura 1. Gramática do Tupã em EBNF, conforme o analisador sintático.

operadores de dois caracteres, como == e >=, precedem os de um caractere, e literais decimais precedem os inteiros. As palavras reservadas não têm expressão própria: o identificador casado é reclassificado por consulta a um dicionário. Um caractere que nenhuma alternativa reconhece produz um erro léxico com linha e coluna.

B. Análise Sintática

O analisador sintático consome os *tokens* e constrói uma árvore sintática abstrata (AST) que representa o dispositivo, suas declarações e seus tratadores de evento. A técnica é a de descendência recursiva preditiva, com um *token* de antecipação e sem retrocesso: cada não-terminal da Fig. 1 é um método do analisador. A precedência das expressões é obtida por uma cadeia de métodos, um por nível, com laços que produzem associatividade à esquerda. O primeiro erro sintático interrompe a compilação, sem recuperação. A AST possui 17 tipos de nó, todos com linha e coluna: `Programa`, `DeclVar`, `Evento`, `Atribuicao`, `Se`, `Repetir`, `Esperar`, `Escrever`, `Retornar`, `ExprComando`, `Binaria`, `Unaria`, `Literal`, `Identificador`, `Hardware`, `AcessoMembro` e `Chamada`.

C. Análise Semântica

A análise semântica é o que diferencia o Tupã de um simples tradutor de texto. Além das verificações usuais, como tipos e declaração de variáveis, ela valida o uso dos módulos de hardware: cada módulo expõe um conjunto conhecido de métodos, e chamadas inexistentes são rejeitadas antes de qualquer código ser gerado. A Fig. 3 mostra a mensagem emitida quando o programador chama um método inexistente no módulo `led`: o erro é localizado por linha e coluna, explicado em português e acompanhado da lista de métodos disponíveis, em tom didático.

D. Geração de Código

O gerador percorre a AST validada e emite um programa Arduino completo: as inclusões de bibliotecas necessárias, como a MFRC522 para o leitor NFC [9]; as definições de pinos; a inicialização dos periféricos em `setup()`; e a tradução dos tratadores de evento para verificações dentro de `loop()`. Bibliotecas e pinos só são emitidos quando o programa usa o módulo correspondente. A pinagem não está mais fixa no gerador: vem de um arquivo de placa em JSON (`compilador/placas/esp32_wrover.json`, escolhido pela opção `-placa`), que traz também o FQBN da Arduino CLI e a lista de UIDs autorizados. Para a ESP32-WROVER-DEV, os LEDs verde e vermelho ficam nos GPIO 25 e 26, o botão 1 no GPIO 4 com *pull-up* interno, o sensor infravermelho no GPIO 34 e o leitor NFC com SS no GPIO 5 e RST no GPIO 22, valores conferidos em bancada. O *debounce* usa `millis()`, sem `delay()`, com janela de 50 ms e ação na borda. A chamada `nfc.ler()` espera até 3 s por um cartão e, sem cartão, devolve um resultado inválido; ao fim da leitura o cartão é encerrado com `PICC_HaltA` e `PCD_StopCrypto1`. A validação compara o UID lido com a lista de UIDs autorizados da placa. Em `setup()`, o código verifica o registrador de versão do RC522 e avisa pelo Serial se o leitor não responde. O compilador grava o resultado em `<pasta>/<pasta>.ino`, formato exigido pela Arduino CLI, e inclui no cabeçalho a origem `.tupa` e a ligação entre blocos e linhas do fonte.

E. Ambiente Web

O projeto inclui um *playground* que expõe o compilador no navegador. Para o código digitado, ele mostra a lista de *tokens*, a AST, os erros, com fase, linha e coluna, e o C++ gerado, além de métricas de linhas. A arquitetura é um serviço FastAPI, que reutiliza o pacote do compilador sem duplicar lógica e expõe o ponto `/api/compilar`, e um *front-end* estático em React com editor CodeMirror, servido pelo próprio serviço. Como o compilador, o ambiente apenas gera o `.ino`.

F. Plataforma de Hardware

O protótipo utiliza uma placa ESP32-WROVER-DEV [8], um leitor NFC/RFID RC522, controlado pela biblioteca MFRC522 [9], um sensor infravermelho de obstáculo, dois LEDs e um botão. A pinagem usada pelo compilador é a do arquivo de placa `esp32_wrover.json`: LEDs nos GPIO 25 e 26, botão no GPIO 4, sensor infravermelho no GPIO 34 e RC522 em SPI com SS no GPIO 5 e RST no GPIO 22. O custo estimado do hardware é de aproximadamente R$ 120, e todo o software utilizado é de código aberto.

V. Resultados Preliminares

As quatro fases do compilador estão implementadas, e um conjunto de 50 testes automatizados, 41 do compilador e 9 da API do ambiente web, é executado com sucesso. Dois programas são traduzidos de ponta a ponta até um arquivo `.ino`: o exemplo canônico da Listagem 1 (`controle_nfc.tupa`) e o teste de hardware em Tupã (`teste_hardware.tupa`), que usa os eventos novos e `escrever`. Os dois arquivos gerados e o `teste_hardware.ino` escrito à mão compilam

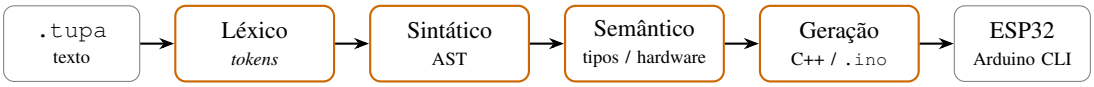


Figura 2. Fluxo de compilação do Tupã, do código-fonte à gravação na placa.

```
$ tupa build controle.tupa

erro semantico - linha 11, coluna 16
    metodo inexistente no modulo 'led'
    disponiveis: verde(), vermelho(), apagar()
    led.roxo()

compilacao interrompida: 1 erro
```

Figura 3. Mensagem de erro semântico emitida pelo compilador.

sem erro com a Arduino CLI para a placa ESP32 Wrover Module (`esp32:esp32:esp32wrover`).

O teste de bancada, de 26/10/2026, foi feito com o firmware escrito à mão (`firmware/teste_hardware.ino`), e a Fig. 4 mostra a montagem e o Monitor Serial. O teste leu três UIDs distintos (`33 7A 87 F7`, `43 A3 6F F5` e `33 8F BD F7`) e registrou o botão e o sensor infravermelho. As imagens, portanto, são do firmware manual: o código gerado pelo compilador ainda *não* foi gravado nem executado na placa.

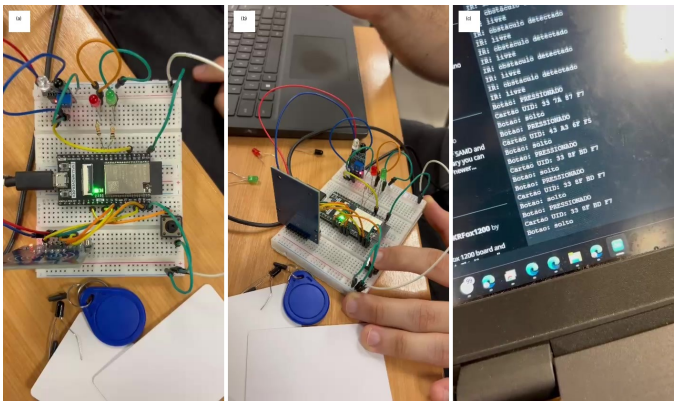


Figura 4. Protótipo em funcionamento: (a) montagem em protoboard com ESP32-WROVER, leitor RC522, sensor IR de obstáculo, botão e LEDs; (b) leitura de uma tag RFID; (c) Monitor Serial registrando eventos do sensor IR, do botão e os UIDs lidos.

A Tabela II compara os programas em Tupã com o C++ gerado e, para o teste de hardware, com o C++ que seria escrito à mão (`teste_hardware.ino`, a referência). O controle de acesso passou de 12 para 100 linhas de código, cerca de 8,3 vezes mais, e o teste de hardware de 23 para 112, cerca de 4,9 vezes. O gerado tem cerca de 2,2 vezes as linhas do manual (112 contra 51), porque inclui *debounce* por `millis()` com detecção de borda, espera de 3 s no NFC, encerramento do cartão, conferência da lista de UIDs e funções auxiliares de LED, que o firmware manual não tem. Nenhuma linha de configuração do hardware precisa ser escrita pelo usuário.

Linhas de código excluem linhas vazias e comentários.

Tabela II
TUPÃ VERSUS C++ GERADO E C++ MANUAL

	controle_nfc		teste_hardware		
Métrica	**Tupã**	**Gerado**	**Tupã**	**Gerado**	**Manual**
Linhas totais	17	126	33	139	67
Linhas de código	12	100	23	112	51
Inclusões de bibliotecas	0	2	0	2	2
Constantes (`#define`)	0	7	0	7	6
Razão C++/Tupã		8,3		4,9	

As duas inclusões são `SPI.h` e `MFRC522.h`. Nos arquivos gerados, as sete constantes são os pinos (cinco no `controle_nfc`, seis no `teste_hardware`), a janela de *debounce* e, no `controle_nfc`, o *timeout* do NFC. No manual, as seis constantes são pinos. Há uma diferença conhecida de comportamento: o firmware manual mantém o LED vermelho aceso enquanto o botão está apertado, e o programa em Tupã o acende por 300 ms a cada aperto.

VI. CONCLUSÃO E TRABALHOS FUTUROS

Este artigo apresentou o Tupã, uma linguagem de domínio específico em português para programação de microcontroladores ESP32, e seu compilador, que percorre as quatro fases clássicas de compilação e gera C++ compatível com o ecossistema Arduino. Os resultados preliminares indicam que a linguagem reduz significativamente a quantidade de código que o usuário precisa escrever e elimina a configuração manual do hardware, enquanto a análise semântica orientada a módulos detecta erros antes da gravação. O protótipo com ESP32-WROVER, RC522, sensor infravermelho de obstáculo, botão e LEDs foi validado em bancada com um firmware escrito à mão, e os programas gerados compilam com a Arduino CLI para a placa-alvo. Falta, porém, gravar o `.ino` gerado na placa e repetir o teste de bancada com ele, de modo que o funcionamento do código gerado no hardware ainda não foi verificado.

Os próximos passos incluem: gravar na placa o código gerado por `controle_nfc.tupa` e `teste_hardware.tupa` e comparar seu comportamento com o do firmware manual; automatizar a compilação e a gravação a partir do comando `tupa build`; ampliar os módulos suportados, como o envio de infravermelho, hoje fora do escopo; documentar a linguagem e o compilador no repositório Git; e, fora do escopo do MVP, permitir a atualização do firmware por Wi-Fi (OTA). Também se pretende avaliar o uso da linguagem em contexto de ensino de sistemas embarcados.

AGRADECIMENTOS

Os autores agradecem ao professor da disciplina de Linguagens Formais e Compiladores do IBMEC pela orientação.

REFERÊNCIAS

[1] A. V. Aho, M. S. Lam, R. Sethi, and J. D. Ullman, *Compilers: Principles, Techniques, and Tools*, 2nd ed. Boston, MA: Addison-Wesley, 2006.

[2] Arduino, "Arduino language reference." [Online]. Available: https://docs.arduino.cc/language-reference/. Acesso em: nov. 2026.

[3] Arduino, "Arduino CLI." [Online]. Available: https://arduino.github.io/arduino-cli/. Acesso em: nov. 2026.

[4] J. Devine, J. Finney, P. de Halleux, M. Moskal, T. Ball, and S. Hodges, "MakeCode and CODAL: intuitive and efficient embedded systems programming for education," *J. Syst. Archit.*, vol. 98, pp. 468–483, 2019.

[5] N. Fraser, "Ten things we've learned from Blockly," in *Proc. IEEE Blocks and Beyond Workshop*, 2015, pp. 49–50.

[6] Flipper Devices, "Flipper Zero." [Online]. Available: https://flipperzero.one/. Acesso em: nov. 2026.

[7] L. H. Noschang, A. Pelz, E. A. Raabe, and A. L. A. Raabe, "Portugol Studio: uma IDE para iniciantes em programação," in *Anais do Workshop sobre Educação em Computação (WEI/CSBC)*, 2014.

[8] Espressif Systems, "ESP32 series datasheet." [Online]. Available: https://www.espressif.com/en/support/documents/technical-documents. Acesso em: nov. 2026.

[9] NXP Semiconductors, "MFRC522: standard performance MIFARE and NTAG frontend," product data sheet.